

Product Requirements Document (PRD)

Project: Autonomous Product Recommendation & Analysis Agent

1. Project Overview

The "Autonomous Product Recommendation Agent" is a stateful, multi-agent system designed to act as a hyper-personalized shopping concierge. Unlike standard static search engines, this system uses LangGraph to dynamically interview the user, deduce unstated needs, autonomously search e-commerce platforms, and create an ephemeral RAG (Retrieval-Augmented Generation) environment allowing users to "chat" with the specifications and reviews of their shortlisted products.

Primary Goal: To build a production-grade AI application demonstrating mastery of Agentic Workflows, LangGraph state management, Tool Calling, and RAG pipelines.

2. Core User Flow

1. **Intake:** User provides an open-ended prompt (e.g., "I'm a final year AI student needing a laptop").
2. **Dynamic Interview (Loop):** The agent analyzes the state and asks a highly targeted question (e.g., budget, compute needs) with multiple-choice options or text input.
3. **Threshold Trigger:** Once the agent determines it has enough constraints (usually 3-5 turns), it exits the interview loop.
4. **Autonomous Search:** The agent converts the profile into optimized search queries and hits e-commerce/search APIs.
5. **Data Ingestion (Ephemeral RAG):** The system scrapes product pages, spec sheets, and reviews for the top 5 results and vectorizes them into temporary memory.
6. **Conversational Analysis:** The user asks comparative questions ("Which has better battery?", "Summarize the negative reviews for the Asus"). The agent answers using citations grounded *only* in the retrieved documents.

3. Technical Architecture & Stack

A. Backend & AI Orchestration (Python)

- **Framework:** FastAPI (High performance, async native, essential for streaming LLM responses).
- **Orchestration:** LangGraph (Crucial for cyclic graphs, state management, and multi-agent routing) & LangChain.
- **LLM Provider:** Google Gemini (via Gemini API) or OpenAI (gpt-4o-mini for speed/cost).
- **Vector Store:** ChromaDB or FAISS (Running in-memory for ephemeral session data).
- **Embeddings:** HuggingFace all-MiniLM-L6-v2 (fast, free) or OpenAI/Gemini embeddings.

B. Tooling & APIs

- **Search/Scraping:** SerpApi (Google Shopping API) or Firecrawl (for extracting clean markdown from product pages and reviews).
- **Structured Output:** Pydantic (Enforcing strict JSON schemas for LLM tool calling).

C. Frontend (Web Interface)

- **Framework:** React (Next.js) or a lightweight alternative like Streamlit/Chainlit if you want to focus purely on backend logic initially.
- **Communication:** Server-Sent Events (SSE) or WebSockets to stream LLM tokens to the UI in real-time.

4. System Modules & LangGraph Framework

Module 1: The Dynamic Interviewer (LangGraph State Machine)

This is a cyclic graph that continues until a condition is met.

- **State Object (TypedDict):**
 - `initial_prompt`: String
 - `chat_history`: List of messages
 - `extracted_preferences`: Dictionary (e.g., `{budget: null, OS: null, use_case: "AI/ML"}`)
 - `is_profile_complete`: Boolean
- **Nodes:**
 - `Analyzer_Node`: Looks at `chat_history` and updates `extracted_preferences`.
 - `Question_Generator_Node`: If `is_profile_complete` is False, generates the next logical question and multiple-choice options.
- **Conditional Edge:** If `is_profile_complete == True`, route to Module 2. Else, loop back to user input.

Module 2: The Orchestrator & Tool Caller

- **Node (Search_Agent):** Takes the finalized `extracted_preferences`.
- **Action:** Uses LLM Tool Calling to map preferences to an API payload. Triggers SerpApi or custom scrapers to fetch the top 5-6 products with their URLs.

Module 3: Ephemeral RAG Pipeline

- **Action:** For the 5 fetched products, trigger an async scraping task (Firecrawl) to pull product specs and the top 20 user reviews.
- **Chunking:** Split the text (LangChain RecursiveCharacterTextSplitter).
- **Indexing:** Embed and store in a temporary ChromaDB collection unique to the user's `session_id`.

Module 4: The Analytical Chatbot (RAG Agent)

- **Node (Comparison_Agent):** A RAG-enabled agent.
- **Behavior:** When the user asks "Compare the GPUs," this agent queries the ephemeral ChromaDB, retrieves the exact spec chunks, and generates a comparative markdown

table citing its sources.

5. "Production-Ready" Requirements (Non-Functional)

To stand out to recruiters, the project must handle edge cases:

1. **Hallucination Prevention:** The Comparison_Agent system prompt MUST strictly enforce: *"Answer ONLY using the provided context. If the spec is missing, state 'Information not available in the scraped data'."*
2. **Streaming:** The UI must stream chunks of text. Users will not wait 15 seconds for a complete JSON payload.
3. **Graceful API Degradation:** If SerpApi fails, the system should catch the timeout and ask the user to refine their query or notify them of network issues, rather than crashing the FastAPI server.
4. **Session Management:** LangGraph's check-pointing (using MemorySaver or Postgres) must be implemented so users can refresh the page without losing their chat history or RAG database.

6. Development Roadmap (Phases)

- **Phase 1: The CLI Prototype (Weeks 1-2)**
 - Set up Python environment.
 - Define LangGraph State and build the Dynamic Interviewer loop.
 - Test in the terminal using basic input() commands.
- **Phase 2: Tooling & RAG (Weeks 3-4)**
 - Integrate SerpApi for fetching real products.
 - Implement Firecrawl/BeautifulSoup to scrape text.
 - Build the in-memory ChromaDB pipeline.
- **Phase 3: Backend API (Week 5)**
 - Wrap the LangGraph workflow in FastAPI endpoints (/start_session, /chat, /get_products).
 - Implement SSE for streaming responses.
- **Phase 4: Frontend & Deployment (Week 6)**
 - Build a clean UI (React/Tailwind).
 - Containerize with Docker.
 - Deploy Backend to Render/Railway and Frontend to Vercel.